

基于多智能体协作的自动化代码审查与缺陷修复系统

详细设计说明文档 (DeepSeek 版)

版本: v2.0

日期: 2026-05-25

作者: Egor-wang

仓库: <https://github.com/Egor-wang/multi-agent-code-review> LLM 选型: DeepSeek-V3 / DeepSeek-Coder-V2

一、系统总体定位与核心价值

1.1 问题定义

当前代码审查 (Code Review) 面临三重困境:

困境	表现	根因
专家瓶颈 一致性差 反馈滞后	资深开发者审查耗时占开发周期 30%+ 不同审查者标准不一，漏检率高 发现问题时缺陷已扩散至下游	领域知识集中于少数人 依赖个人经验，缺乏标准化 审查发生在编码完成后

1.2 系统定位

不是一个” 更好的 Lint 工具”，而是一个” 可协作的虚拟审查团队”

本系统通过**多智能体协作架构**，模拟真实代码审查团队中不同角色（安全专家、架构师、测试工程师）的专业分工与协作流程，实现：

- **分层审查**: 静态规则 -> 语义分析 -> 动态验证 -> 自动修复
- **知识沉淀**: 将历史审查经验转化为可检索、可复用的组织资产
- **人机协同**: AI 处理模式化问题, 人类聚焦架构决策与业务逻辑

1.3 核心价值主张

传统方案	本系统
单模型补全	-> 多 Agent 专业分工
黑盒建议	-> 白盒推理过程
通用规则	-> 项目专属知识
一次性审查	-> 迭代式修复验证

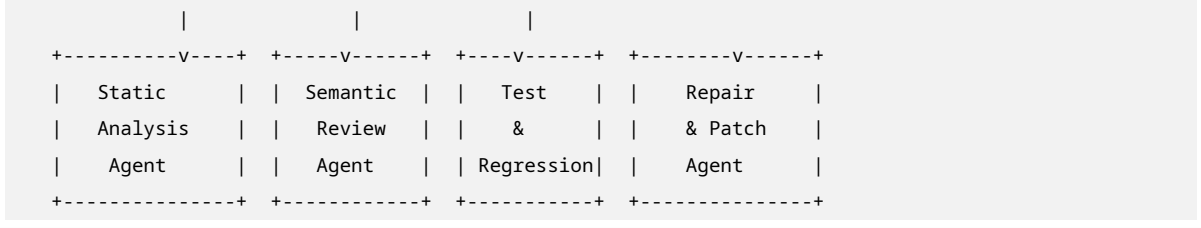
二、多智能体角色划分与职责表

2.1 Agent 体系总览

```

classDiagram
    class ReviewOrchestrator {
        <<component>>
    }
    
```

The diagram shows a single component box for the Review Orchestrator. The box is divided into three horizontal sections. The top section is empty. The middle section contains the text "Review Orchestrator". The bottom section contains the text "(审查协调器 / 中央调度)".



2.2 各 Agent 详细设计

Agent 1: Static Analysis Agent (静态分析 Agent)

维度	说明
核心职责	基于语法与结构规则，执行模式匹配式缺陷检测
审查维度	安全性（注入漏洞、敏感信息泄露）、可维护性（圈复杂度、重复代码）、规范性（命名、格式）
输入	代码变更 Diff、文件 AST、项目语言类型
输出	结构化缺陷列表（位置、规则 ID、严重程度、修复模板）
技术栈	Tree-sitter（AST 解析）、Semgrep（模式匹配）、Pylint/ESLint（语言特定规则）
协作关系	向 Orchestrator 提交初步缺陷候选；为 Semantic Agent 提供可疑位置标记
设计理由	静态分析具有确定性、低误报、可解释的优势，适合作为审查第一道防线

Agent 2: Semantic Review Agent (语义审查 Agent)

维度	说明
核心职责	理解代码语义与业务逻辑，识别设计模式违规、逻辑错误、API 误用
审查维度	正确性（空指针、并发竞态）、性能（算法复杂度、N+1 查询）、架构一致性
输入	AST + CFG（控制流图）、变更上下文（相邻函数、调用链）、RAG 检索结果（历史相似 Bug）
输出	语义级审查意见（含根因分析、影响面评估、修复建议）
技术栈	DeepSeek-V3（通用语义理解）+ DeepSeek-Coder-V2（代码专用）+ RAG 检索 + 程序分析（数据流分析）
协作关系	接收 Static Agent 的可疑点进行深度分析；向 Repair Agent 输出修复上下文
设计理由	弥补静态分析无法理解业务逻辑的短板，是” AI 价值”的核心体现

Agent 3: Test & Regression Agent (测试与回归 Agent)

维度	说明
核心职责	评估变更的测试覆盖度，生成缺失测试，预测回归风险

Table 4 – continued

维度	说明
审查维度	测试覆盖率、边界条件、回归风险、兼容性影响
输入	变更代码、现有测试套件、代码覆盖率报告、CI 历史数据
输出	测试缺口报告、生成的测试用例（代码）、回归风险评级
技术栈	Coverage.py/JaCoCo（覆盖率）、DeepSeek-Coder-V2（测试生成）、变异测试（Mutation Testing）
协作关系	为 Repair Agent 提供测试验证标准；向 Orchestrator 报告”可合并性”评估
设计理由	审查的终极目标是保证质量，测试是质量的可量化证明

Agent 4: Repair & Patch Agent (修复与补丁 Agent)

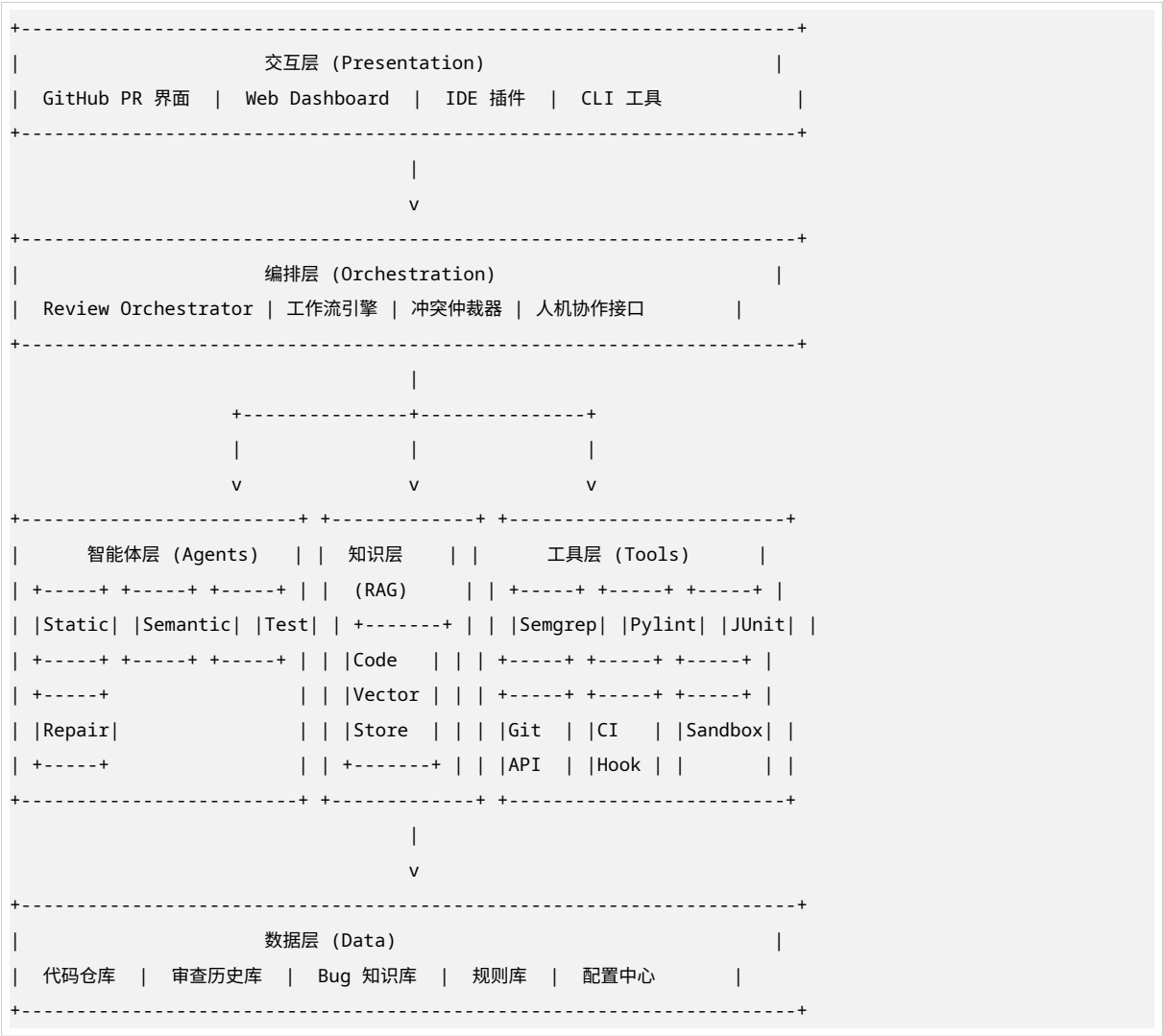
维度	说明
核心职责	基于审查意见生成可应用的代码修复补丁，并验证修复有效性
审查维度	修复正确性、最小侵入性、风格一致性
输入	原始代码、审查意见（含位置与类型）、项目编码规范、测试用例
输出	统一 Diff 格式的修复补丁、修复解释（自然语言）、验证结果
技术栈	DeepSeek-Coder-V2（代码生成）、Tree-sitter（确保语法正确性）、沙箱执行验证
协作关系	接收 Semantic Agent 的修复建议；请求 Test Agent 验证修复；向 Orchestrator 提交修复候选
设计理由	从”发现问题”到”解决问题”闭环，体现工程实用性

Agent 5: Review Orchestrator (审查协调器)

维度	说明
核心职责	workflow 调度、冲突消解、质量门控、人机协作接口
核心功能	1. 变更分类与 Agent 分发 2. 结果汇总与去重 3. 冲突仲裁 4. 迭代控制 5. 人工介入点管理
输入	PR 元数据、各 Agent 输出、人工反馈、项目策略配置
输出	统一审查报告、质量门控决策（通过/需修改/阻断）、待人工确认项
技术栈	LangGraph（状态机编排）、规则引擎（冲突消解）、优先级队列
协作关系	所有 Agent 的中央调度节点；与 GitHub API 交互更新 PR 状态
设计理由	多 Agent 系统的”操作系统”，决定协作效率与系统可靠性

三、系统总体架构图

3.1 分层架构（文字版）



3.2 Mermaid 架构图



```
SR[Semantic Review Agent]
TR[Test & Regression Agent]
RP[Repair & Patch Agent]
end
```

```
subgraph 知识层
  RAG[RAG 检索引擎]
  VS[(Vector Store)]
  BK[(Bug Knowledge)]
end
```

```
subgraph 工具层
  SE[Semgrep]
  PI[PyLint/ESLint]
  JT[测试框架]
  GH_API[GitHub API]
  SB[沙箱验证]
end
```

```
GH --> ORCH
ORCH --> WF
WF --> SA & SR & TR
SA --> SE & PI
SR --> RAG
RAG --> VS & BK
TR --> JT
SA & SR & TR --> ARB
ARB --> RP
RP --> SB
RP --> HCI
HCI --> GH_API
GH_API --> GH
```

四、Agent 协作工作流（状态机设计）

4.1 核心工作流：PR 审查流水线

```
stateDiagram-v2
    [*] --> 接收 PR: GitHub Webhook

    接收 PR --> 变更分析: Orchestrator
    变更分析 --> 分类决策: 提取 Diff/AST/元数据

    state fork_state <<fork>>
        分类决策 --> fork_state: 按变更类型分发

        fork_state --> 静态审查: Static Agent
        fork_state --> 语义审查: Semantic Agent
        fork_state --> 测试评估: Test Agent
```

```
state join_state <<join>>
静态审查 --> join_state
语义审查 --> join_state
测试评估 --> join_state

join_state --> 冲突仲裁: Orchestrator

state 冲突仲裁 {
    [*] --> 结果汇总
    结果汇总 --> 冲突检测: 多 Agent 意见冲突?
    冲突检测 --> 规则仲裁: 是 (按优先级/置信度)
    冲突检测 --> 人工介入: 无法自动仲裁
    规则仲裁 --> [*]
    人工介入 --> [*]: 等待开发者确认
}

冲突仲裁 --> 质量门控: 生成统一报告

state 质量门控 {
    [*] --> 严重缺陷?
    严重缺陷? --> 阻断合并: 是
    严重缺陷? --> 允许合并: 否
    阻断合并 --> 自动修复: 触发 Repair Agent
    自动修复 --> 验证修复: Test Agent
    验证修复 --> 重新审查: 闭环
    重新审查 --> 严重缺陷?
    允许合并 --> [*]
}

质量门控 --> 输出报告: 结构化 JSON/Markdown
输出报告 --> [*]: 更新 PR 状态
```

4.2 迭代修复循环（关键创新点）

```
第 1 轮审查
Static Agent 发现: SQL 注入风险 @ line 45
Semantic Agent 确认: 用户输入未过滤直接拼接
Test Agent 报告: 缺少安全测试用例

-> Orchestrator 决策: 阻断合并, 触发修复

第 1 轮修复
Repair Agent 生成: 参数化查询补丁
Test Agent 验证: 新增测试通过, 回归测试通过

-> Orchestrator 决策: 进入重审

第 2 轮审查 (Re-review)
Static Agent: 注入风险已消除
Semantic Agent: 修复符合项目规范, 但建议添加输入长度校验
Test Agent: 新测试覆盖边界条件
```

-> Orchestrator 决策：降级为警告，允许合并，附改进建议

4.3 失败回退策略

失败场景	回退策略	保障机制
Agent 执行超时	标记为” 未完成”，人工介入	超时配置 (30s/Agent)
LLM API 不可用	降级为纯静态分析模式	本地规则库兜底
工具调用失败	跳过该 Agent，其他 Agent 继续	熔断机制
修复验证失败	回滚修复，保留审查意见	沙箱隔离执行
冲突无法仲裁	提升为” 需人工决策”	人工优先级最高

五、RAG 与知识增强设计

5.1 三层知识库架构



5.2 知识编码与检索策略

向量化策略（避免幻觉的关键）：

知识类型	编码方式	检索触发条件
代码规范	规范文本 + 正/反例代码片段	Semantic Agent 审查时
历史 Bug	Bug 描述 + 问题代码 + 修复 Diff + 根因标签	代码相似度匹配
架构约束	ADR 文档 + 关键接口定义	涉及架构变更时
审查经验	审查意见 + 采纳结果	同类变更模式识别

防幻觉机制：

- 1. 检索结果约束：LLM 生成时必须引用检索到的具体规范条目或历史案例 ID
- 2. 验证闭环：Repair Agent 生成的修复必须通过静态分析工具验证，确保语法正确

- 3. 置信度阈值：检索相似度 <0.7 时，标记为”推测性建议”，不自动修复
- 4. 人工确认点：涉及安全修复时，必须人工确认后方可应用

5.3 知识沉淀流程



六、工具调用与工程集成

6.1 工具调用架构 (Tool Use)



6.2 工具调用协议

每个工具封装为统一接口：

```
class ToolInterface:
    def plan(self, context: ReviewContext) -> ToolPlan:
        # 基于上下文决定是否需要调用
        pass

    def execute(self, plan: ToolPlan) -> RawResult:
        # 执行工具, 返回原始输出
```



```
pass

def parse(self, raw: RawResult) -> StructuredResult:
    # 解析为 Agent 可理解的结构化数据
    pass

def validate(self, result: StructuredResult) -> bool:
    # 验证结果有效性 (防止工具误报/执行失败)
    pass
```

6.3 与 GitHub/CI 的深度集成



七、评估与可信度设计

7.1 多维度评估体系

维度	指标	测量方法
审查准确率	精确率 (Precision) / 召回率 (Recall)	与人工审查结果对比, F1-score
修复成功率	语法正确率 / 测试通过率 / 语义等价率	沙箱执行 + 变异测试
误报率	无效审查意见占比	开发者标记 "无用" / "误报"
效率提升	平均审查时间 / 发现问题阶段前移率	与纯人工审查对比
可解释性	审查意见含根因分析的比例	人工评分

7.2 可信度分级机制

阻断级 (Blocker): 静态分析确认 + Semantic Agent 高置信度 -> 必须修复
警告级 (Warning): 单 Agent 发现, 未交叉验证 -> 建议修复, 需人工确认
建议级 (Suggestion): 模式匹配/推测性建议 -> 参考性意见

7.3 可解释性设计 (Explainability)

每条审查意见必须包含:

```
{
  "issue_id": "SEC-2024-001",
  "type": "SQL 注入",
  "severity": " 阻断",
  "location": "src/auth.py:45",
  "root_cause": " 用户输入`username`直接拼接至 SQL 语句, 未经过参数化处理",
  "evidence": {
    "code_snippet": "query = f'SELECT * FROM users WHERE name = \"{username}\"",
    "ast_path": "FunctionDef:login -> Assign:query -> JoinedStr",
    "similar_bug": "Issue#234: 2023-08 生产环境 SQL 注入事件"
  },
  "fix_suggestion": {
    "diff": "@@ -45,3 +45,3 @@...",
    "explanation": " 使用参数化查询, 将用户输入与 SQL 逻辑分离, 从根本上消除注入风险",
    "references": ["OWASP-Cheatsheet: SQL Injection Prevention"]
  },
  "confidence": 0.95,
  "verification": {
    "static_check": " 通过",
    "test_result": " 新增测试 test_sql_injection_prevention: 通过"
  }
}
```

八、关键技术选型与设计理由

技术组件	选型	设计理由
多智能体框架	LangGraph	状态机模型天然适合审查流程的复杂分支与回退; 与 LangChain 生态无缝集成; 社区活跃, 学术认可度高
LLM	DeepSeek-V3 / DeepSeek-Coder-V2	国产顶尖模型, 代码理解能力对标 GPT-4; 支持 128K 上下文; API 成本仅为 OpenAI 1/10; 中文场景优化
代码解析	Tree-sitter	增量解析速度快, 支持多语言, AST 精度高, 适合构建代码知识库
向量数据库	Chroma (初期) -> Milvus (扩展)	轻量起步, 后期可横向扩展; 支持混合检索
静态分析	Semgrep + 语言特定 Linters	Semgrep 规则可自定义, 适合项目特定模式; Linters 提供确定性基础检查
后端服务	FastAPI	异步支持 Webhook 高并发; 类型提示提升代码质量; 生态成熟

Table 10 – continued

技术组件	选型	设计理由
部署	Docker + GitHub Actions	容器化保证环境一致；CI/CD 实现审查流程自动化

九、与工业方案对比优势

维度	SonarQube	CodeRabbit	本系统
架构模式	单体式规则引擎	单模型 + 模板	多 Agent 协作，角色分工
审查深度	语法/结构层	语义层 (LLM)	分层递进：静态-> 语义-> 测试-> 修复
知识能力	预置规则库	通用 LLM 知识	RAG 增强，项目专属知识沉淀
修复能力	无	简单建议	生成可验证的 Diff 补丁
可解释性	规则 ID 引用	自然语言解释	结构化根因分析 + 证据链
人机协作	被动报告	被动评论	主动仲裁 + 迭代修复 + 人工介入点
开源可控	部分开源	闭源商业	全栈开源，可深度定制

核心差异化：工业方案追求”通用性”，本系统追求 “领域深度 + 工程可控性”，更适合作为科研探索与垂直场景落地的原型。

十、保研展示设计亮点总结

10.1 一句话概括每个模块

模块	一句话说明
Review Orchestrator	“审查团队的项目经理，负责任务分发与冲突协调”
Static Analysis Agent	“语法专家，用确定性规则快速筛查明显缺陷”
Semantic Review Agent	“架构专家，理解业务逻辑，发现深层设计问题”
Test & Regression Agent	“质量守门员，确保变更不会破坏现有功能”
Repair & Patch Agent	“自动修复工程师，生成可验证的代码补丁”
RAG 知识库	“组织经验沉淀器，让历史教训不再重复发生”

10.2 架构图展示建议

面试时手绘或展示：

[展示要点]

- 1. 先画分层架构（交互层-> 编排层-> 智能体层-> 知识层-> 工具层）
- 2. 强调" 分层审查" 理念：静态规则（确定性）-> 语义理解（智能性）-> 测试验证（可靠性）-> 自动修复（闭环性）
- 3. 用颜色区分：蓝色 = 确定性组件，橙色 =AI 组件，绿色 = 人工介入点

10.3 Demo 演示流程（5 分钟版本）

- 第 1 分钟：展示系统架构图，说明" 为什么是多 Agent 而非单模型"
- 第 2 分钟：输入一段含 SQL 注入的代码变更，展示 Static Agent 秒级检出
- 第 3 分钟：展示 Semantic Agent 的根因分析（引用 OWASP 规范 + 历史 Bug 案例）
- 第 4 分钟：展示 Repair Agent 生成参数化查询补丁，Test Agent 验证通过
- 第 5 分钟：展示迭代审查：修复后 Re-review 确认安全，系统允许合并

10.4 保研面试高频问题预设

Q: 为什么不用一个强大的 LLM 直接做审查？

A: 单一 LLM 存在幻觉、成本高、不可控的问题。多 Agent 架构将确定性任务（静态分析）交给规则引擎，将智能任务（语义理解）交给 LLM，实现可控成本下的最大智能。

Q: 如何防止 AI 生成错误的修复？

A: 三层防护：1. 修复必须通过静态分析语法检查 2. 必须在沙箱中通过测试验证 3. 安全类修复必须人工确认。我们追求“可验证的自动化”而非“全自动”。

Q: 与 Copilot/CodeRabbit 的区别？

A: 它们是”代码补全工具”或”评论机器人”，我们是”审查 workflow 系统”。核心差异在于状态机编排、多轮迭代、知识沉淀、闭环修复。

附录：项目里程碑规划

阶段	周期	目标	可展示成果
M1: 骨架搭建	第 1-2 周	项目结构、README、Orchestrator 框架	架构文档 + 空转状态机
M2: 静态审查	第 3-4 周	Static Agent + Semgrep 集成	输入代码-> 输出缺陷列表
M3: 语义增强	第 5-6 周	Semantic Agent + RAG 检索	含根因分析的审查意见
M4: 测试闭环	第 7-8 周	Test Agent + Repair Agent	生成测试 + 修复补丁
M5: 系统集成	第 9-10 周	GitHub Webhook + Dashboard	完整 PR 审查 Demo
M6: 评估优化	第 11-12 周	对比实验、指标优化	实验报告 + 论文初稿

本文档作为系统设计基准，后续开发中可根据实际约束调整技术细节，但核心架构与设计理念保持不变。